

UNIVERSIDADE DE MOGI DAS CRUZES

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

LETICIA GABRIELA FERREIRA

LOGIN SEGURO

Mogi das Cruzes

2026

LETICIA GABRIELA FERREIRA

LOGIN SEGURO

Trabalho desenvolvido para apresentação à disciplina de Aplicativos Web, do curso de Sistemas de Informação da Universidade de Mogi das Cruzes como requisito parcial para aprovação na disciplina.

Professor: Alessandro Horas

Mogi das Cruzes

2026

Sumário

1 INTRODUÇÃO.....	4
2 TECNOLOGIAS UTILIZADAS	4
3 ESTRUTURA DO SISTEMA.....	5
3.1 Fluxo de uma requisição.....	5
3.2 Entidade Usuário	5
4 INTEGRAÇÃO COM O MONGODB ATLAS.....	6
4.1 Conexão segura	6
4.2 Coleções.....	7
5 AUTENTICAÇÃO E AUTORIZAÇÃO.....	8
5.1 Cadastro e criptografia da senha.....	8
5.2 Login e logout	9
5.3 Perfis de acesso	9
6 INTERFACE E TEMAS.....	11
7 PRINCIPAIS DECISÕES DE DESIGN	12
8 COMO EXECUTAR O SISTEMA.....	12
9 CONTROLE DE VERSÃO	13
REFERÊNCIAS	14

1 INTRODUÇÃO

Este documento técnico apresenta os parâmetros e funcionalidades do sistema Login Seguro, uma aplicação web desenvolvida com Java Spring Boot, Thymeleaf e MongoDB Atlas. Esse sistema permite o cadastro, o login e o logout de usuários, além do controle de acesso às páginas de acordo com o perfil de cada usuário.

O objetivo desta documentação é explicar a estrutura do sistema, a integração com o MongoDB Atlas e as principais decisões de design, servindo de apoio para futuras adaptações.

2 TECNOLOGIAS UTILIZADAS

As tecnologias foram escolhidas com base no conteúdo trabalhado na disciplina e nos requisitos da atividade. O Quadro 1 resume cada uma delas e a sua função no sistema.

Quadro 1 – Tecnologias utilizadas no sistema

Tecnologia	Descrição do uso
Java 17	Linguagem de programação do projeto.
Spring Boot 3.4	Framework que inicia a aplicação e configura o servidor web.
Spring Security	Responsável pela autenticação, pela autorização e pela criptografia das senhas.
Spring Data MongoDB	Faz a comunicação entre as classes Java e o banco de dados.
Spring Session MongoDB	Armazena as sessões dos usuários no banco de dados.
Thymeleaf	Motor de templates que gera as páginas HTML no servidor.
Bootstrap 5	Biblioteca de estilos usada nas páginas.
MongoDB Atlas	Banco de dados em nuvem onde ficam os usuários e as sessões.
Maven	Gerenciador de dependências e de execução do projeto.
Git e GitHub	Controle de versão e hospedagem do código.

3 ESTRUTURA DO SISTEMA

O sistema segue o padrão MVC (Model, View, Controller), separando os dados, as regras de negócio e as páginas. O código Java está organizado em cinco pacotes, cada um com uma responsabilidade, como mostra o Quadro 2.

Quadro 2 – Organização dos pacotes

Pacote	Classes	Responsabilidade
entity	Usuario, Perfil	Representam os dados gravados no banco.
repository	UsuarioRepository	Acesso à coleção de usuários no MongoDB.
service	UsuarioService, AutenticacaoService	Regras de negócio e busca do usuário no momento do login.
controller	AutenticacaoController, HomeController, AdminController	Recebem as requisições e devolvem as páginas.
config	SecurityConfig, TemaConfig, AdminInicialConfig	Configurações de segurança, de tema e criação do primeiro administrador.

3.1 Fluxo de uma requisição

Para o fluxo de requisição, quando o usuário acessa uma página, a requisição passa primeiro pelo Spring Security, que verifica se ele está autenticado e se o seu perfil tem permissão para aquela rota. Se tiver, o controller é chamado, busca os dados necessários no service e devolve o nome do template. Por fim, o Thymeleaf monta a página HTML e a envia ao navegador.

3.2 Entidade Usuário

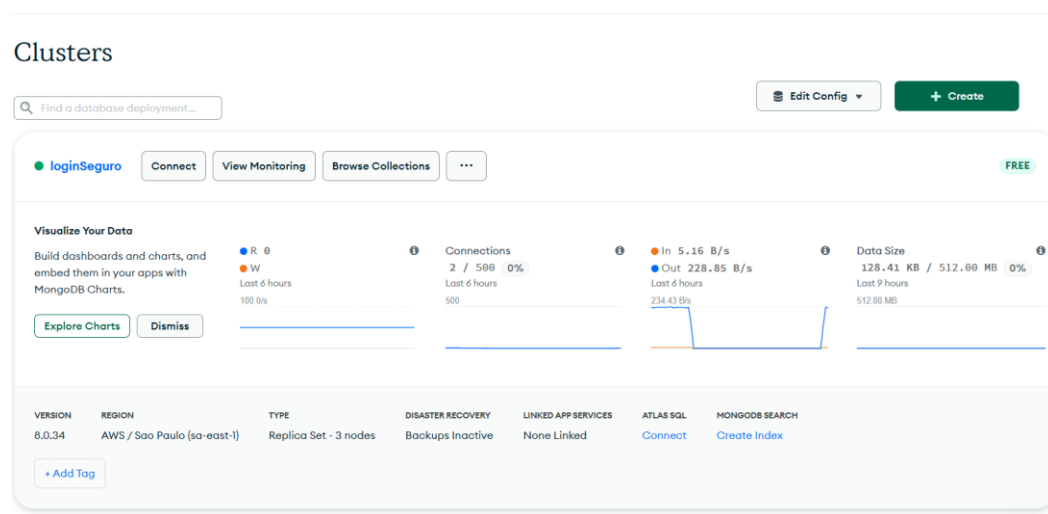
A classe Usuário possui os campos id, nome, e-mail, senha e perfil. Os campos nome, e-mail e senha possuem anotações de validação, que impedem o cadastro com dados em branco, com e-mail em formato inválido ou com senha menor que oito

caracteres. O campo e-mail possui índice único, o que impede dois cadastros com o mesmo endereço.

4 INTEGRAÇÃO COM O MONGODB ATLAS

O MongoDB Atlas é o serviço de banco de dados em nuvem do MongoDB. Para este projeto foi criado um cluster gratuito, um usuário de banco de dados e uma regra de acesso de rede liberando o endereço IP da máquina de desenvolvimento.

Figura 1 – Cluster criado no MongoDB Atlas



4.1 Conexão segura

A conexão é feita por meio de uma string de conexão fornecida pelo próprio Atlas, que contém o usuário e a senha do banco. Por segurança, essa string não fica escrita no código. Ela é lida de um arquivo chamado `.env`, que fica apenas na máquina local e está listado no arquivo `.gitignore`, portanto não é enviado ao GitHub.

No repositório existe apenas o arquivo `.env.example`, com valores fictícios, que serve de modelo. O arquivo `application.properties` busca o valor da variável, como mostrado a seguir.

```
spring.config.import=optional:file:.env[.properties]
```

```
spring.data.mongodb.uri=${MONGODB_URI}
```

```
spring.data.mongodb.database=loginseguro
```

```
spring.data.mongodb.auto-index-creation=true
```

4.2 Coleções

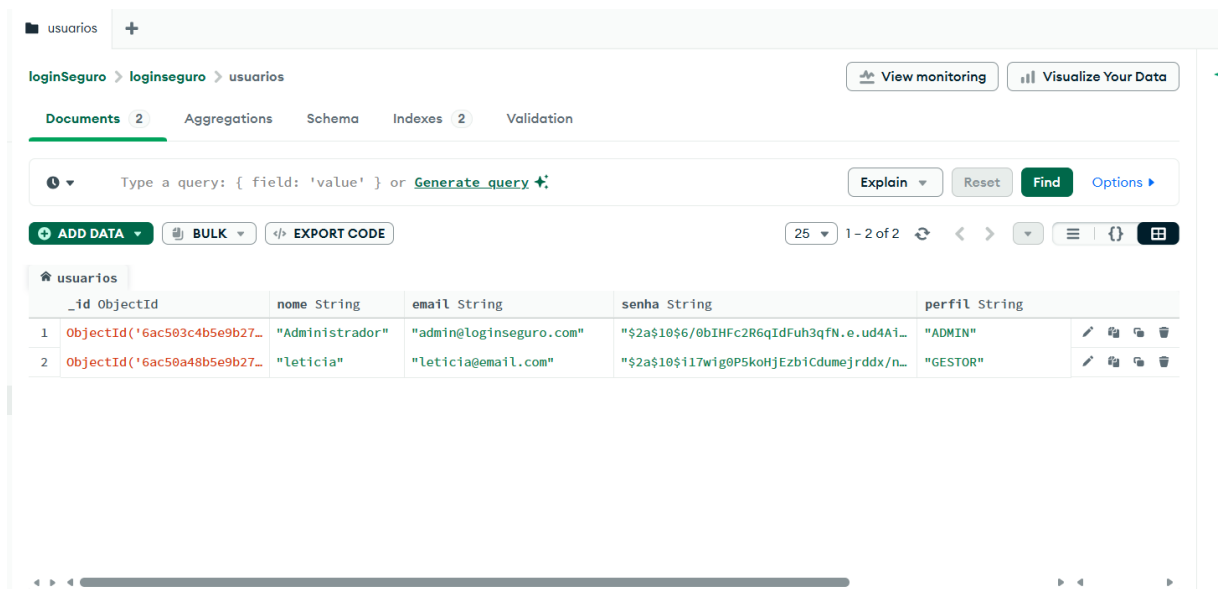
O sistema utiliza o banco de dados loginseguro, com duas coleções, descritas no Quadro 3. As duas são criadas automaticamente na primeira execução.

Quadro 3 – Coleções do banco de dados

Coleção	Conteúdo	Funcionalidade
usuarios	Nome, e-mail, senha criptografada e perfil de cada usuário.	UsuarioRepository (Spring Data MongoDB).
sessoes	Sessões dos usuários que estão autenticados.	Spring Session MongoDB.

O acesso à coleção usuarios é feito pela interface UsuarioRepository, que estende MongoRepository. Com isso, as operações de salvar, listar, buscar e excluir já ficam disponíveis, sem a necessidade de escrever consultas.

Figura 2 – Coleção usuarios no MongoDB Atlas



loginSeguro > loginseguro > usuarios					
Documents 2 Aggregations Schema Indexes 2 Validation					
Type a query: { field: 'value' } or Generate query					
ADD DATA BULK EXPORT CODE 25 1 - 2 of 2					
usuarios					
	_id ObjectId	nome String	email String	senha String	perfil String
1	ObjectId('6ac593c4b5e9b27...')	"Administrador"	"admin@loginseguro.com"	"\$2a\$10\$6/8bIHFc2R6qIdFuh3qfN.e.ud4Ai..."	"ADMIN"
2	ObjectId('6ac59a48b5e9b27...')	"Letícia"	"leticia@email.com"	"\$2a\$10\$117wig0P5koHjEzbiCdumejrddx/n..."	"GESTOR"

4.3 Armazenamento das sessões

Por padrão, o Spring guarda as sessões na memória do servidor, para atender ao requisito de armazenar as sessões no MongoDB Atlas, foi adicionada a dependência Spring Session MongoDB. Com essa dependência, cada login gera um

documento na coleção sessoes, que é removido quando o usuário sai do sistema ou quando a sessão expira, após 30 minutos sem uso.

Figura 3 – Coleção sessoes no MongoDB Atlas

	_id String	created Date	accessed Date	interval String	principal Mixed	expireAt Date	attr Binary
1	"6c4748a3-4bef-43d8-813a-..."	ISODate("2026-10-06T14:25:..."	ISODate("2026-10-06T14:25:..."	"PT30M"	null	ISODate("2026-10-06T14:55:..."	Binary.createFromB...
2	"44dc3e98-7b84-4a26-8098-..."	ISODate("2026-10-06T14:43:..."	ISODate("2026-10-06T14:43:..."	"PT30M"	null	ISODate("2026-10-06T15:13:..."	Binary.createFromB...
3	"bb9a5c32-2e48-4f3c-88e7-..."	ISODate("2026-10-06T14:49:..."	ISODate("2026-10-06T14:49:..."	"PT30M"	null	ISODate("2026-10-06T15:19:..."	Binary.createFromB...
4	"ccf66653-7454-4858-8483-..."	ISODate("2026-10-06T14:49:..."	ISODate("2026-10-06T14:49:..."	"PT30M"	null	ISODate("2026-10-06T15:19:..."	Binary.createFromB...
5	"b912d89a-e472-4f62-abc8-..."	ISODate("2026-10-06T14:49:..."	ISODate("2026-10-06T14:58:..."	"PT30M"	"leticia@email.com"	ISODate("2026-10-06T15:28:..."	Binary.createFromB...

5 AUTENTICAÇÃO E AUTORIZAÇÃO

5.1 Cadastro e criptografia da senha

Durante o cadastro, o usuário informa nome, e-mail e senha. Esses dados são validados e, se o e-mail já existir, uma mensagem de erro é exibida e o processo não segue. Antes de salvar, a senha passa pelo algoritmo BCrypt, por meio da classe BCryptPasswordEncoder do Spring Security. Assim, o banco de dados guarda apenas o hash da senha, e nunca a senha original.

Figura 4 – Tela de cadastro com validação

Criar conta

Nome
Leticia

E-mail
leticia@email.com
E-mail já cadastrado

Senha

Cadastrar

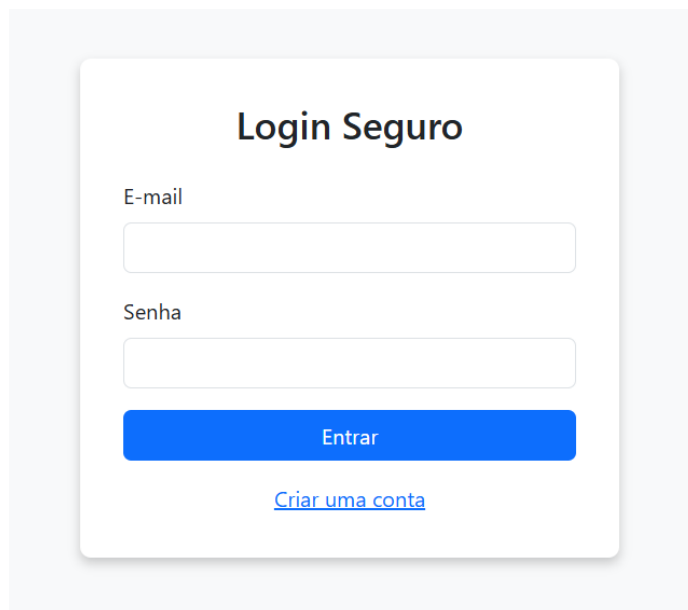
[já tenho uma conta](#)

5.2 Login e logout

O login utiliza o formulário do Spring Security, com uma página personalizada e ao receber o e-mail e a senha, o Spring chama a classe `AutenticacaoService`, que busca o usuário no banco pelo e-mail. Em seguida, o próprio Spring compara a senha digitada com o hash armazenado. O logout encerra a sessão e apaga o cookie do navegador.

A proteção contra CSRF, que vem ativa por padrão no Spring Security, foi mantida. O Thymeleaf inclui automaticamente o token de segurança em todos os formulários.

Figura 5 – Tela de login

A imagem mostra uma interface de login segura. No topo, o título "Login Seguro" está centralizado. Abaixo dele, há dois campos de entrada: "E-mail" e "Senha". Cada campo é representado por um retângulo branco com uma borda cinza. Abaixo dos campos, há um botão azul com o texto "Entrar" em branco. Na base da interface, há um link azul com o texto "Criar uma conta".

5.3 Perfis de acesso

Esse sistema possui três perfis, definidos no enum `Perfil`: ADMIN, GESTOR e USUARIO. Todo usuário que se cadastra recebe o perfil USUARIO. Apenas o administrador pode alterar o perfil de outro usuário ou excluí-lo. O Quadro 4 mostra as permissões de cada perfil.

Quadro 4 – Permissões por perfil

Página	Rota	USUARIO	GESTOR	ADMIN
--------	------	---------	--------	-------

Início	/	Sim	Sim	Sim
Painel do gestor	/gestor/painel	Não	Sim	Sim
Usuários	/admin/usuarios	Não	Não	Sim

As regras de acesso ficam concentradas na classe SecurityConfig, conforme o trecho a seguir.

```
.requestMatchers("/login", "/cadastro", "/css/**").permitAll()
```

```
.requestMatchers("/admin/**").hasRole("ADMIN")
```

```
.requestMatchers("/gestor/**").hasAnyRole("ADMIN", "GESTOR")
```

```
.anyRequest().authenticated()
```

Quando um usuário tenta acessar uma página sem permissão, o sistema exibe a página de acesso negado. Além disso, os itens do menu que o perfil não pode acessar não são exibidos.

Figura 6 – Página de gestão de usuários

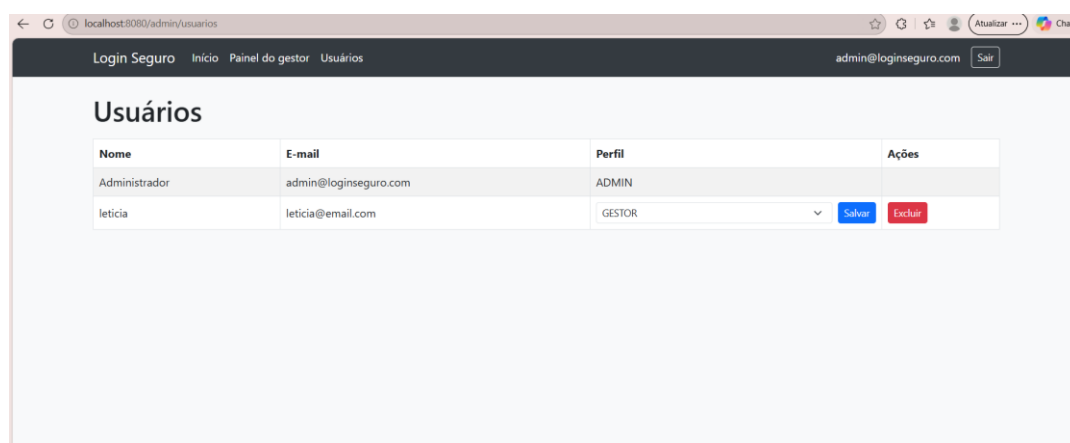


Figura 7 – Página de acesso negado



6 INTERFACE E TEMAS

As páginas de navegação foram construídas com Thymeleaf e Bootstrap. O cabeçalho e o menu de navegação foram escritos uma única vez, no arquivo *fragments/layout.html*, e são reaproveitados em todas as páginas por meio de fragmentos. Dessa forma, uma alteração no menu é refletida em todo o sistema.

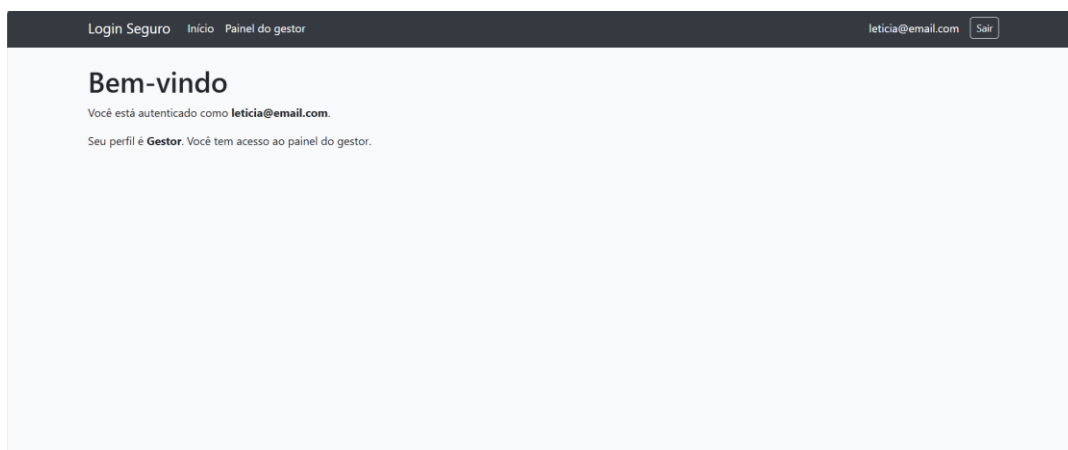
Para permitir a personalização visual, as cores do sistema foram definidas como variáveis CSS. Cada tema é um arquivo na pasta *static/css/temas* contendo apenas essas variáveis. O arquivo *style.css* utiliza as variáveis, sem cores fixas.

O tema em uso e o nome exibido no sistema são definidos no arquivo *application.properties*, pelas propriedades a seguir.

app.nome=Login Seguro

app.tema=padrao

Figura 8 – Página inicial do sistema



7 PRINCIPAIS DECISÕES DE DESIGN

1. Separação em camadas: cada pacote tem uma responsabilidade, o que facilita localizar e alterar o código.
2. Uso do e-mail como identificador de login: é um dado único e que o usuário já conhece.
3. Senha com BCrypt: é o algoritmo recomendado pelo Spring Security e já inclui proteção contra ataques por tabelas de senhas.
4. Perfis em um enum: para criar um novo perfil basta acrescentar um valor e liberar as rotas no SecurityConfig.
5. Perfil definido pelo sistema no cadastro: o usuário não escolhe o próprio perfil, o que evita que alguém se cadastre como administrador.
6. Credenciais fora do código: a string de conexão e a senha do administrador ficam no arquivo `.env`, que não é versionado.
7. Sessões no banco de dados: atende ao requisito da atividade e mantém os usuários autenticados mesmo se a aplicação for reiniciada.
8. Temas por variáveis CSS: permitem mudar a aparência do sistema sem alterar as páginas nem o código Java.

8 COMO EXECUTAR O SISTEMA

1. Para executar o sistema localmente são necessários o Java 17 ou superior, o Git e uma conta no MongoDB Atlas. Os passos são os seguintes.
2. Clonar o repositório do GitHub.
3. Criar um cluster gratuito no MongoDB Atlas, um usuário de banco de dados e liberar o endereço IP.
4. Copiar o arquivo `.env.example` com o nome `.env` e preencher a string de conexão e os dados do administrador.
5. Executar o comando `mvnw.cmd spring-boot:run` no Windows, ou `./mvnw spring-boot:run` no Linux e no macOS.
6. Acessar o endereço `http://localhost:8080` no navegador.

As instruções detalhadas estão no arquivo README do repositório.

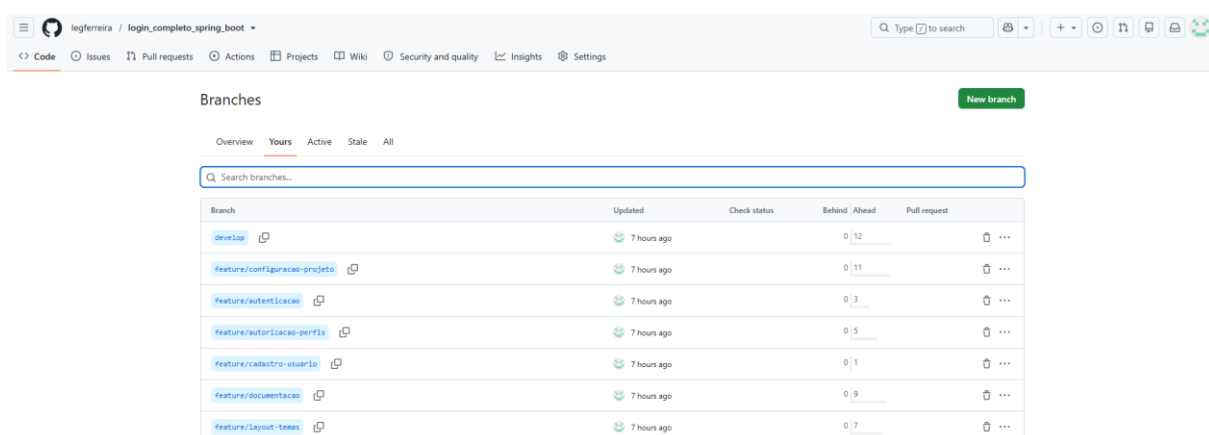
9 CONTROLE DE VERSÃO

Esse projeto foi versionado com Git, seguindo o fluxo Gitflow. A branch main guarda a versão entregue, a branch develop reúne o desenvolvimento e cada funcionalidade foi criada em uma branch feature separada, depois unida à develop. O Quadro 5 lista as branches de funcionalidade.

Quadro 5 – Branches de funcionalidade

Branch	Conteúdo
feature/cadastro-usuario	Entidade, repositório, serviço e tela de cadastro.
feature/autenticacao	Configuração de segurança, login e logout.
feature/autorizacao-perfis	Páginas restritas por perfil e gestão de usuários.
feature/layout-temas	Layout com fragmentos e temas configuráveis.
feature/documentacao	Arquivo README.
feature/configuracao-projeto	Configuração do Maven e arquivo de exemplo de ambiente.

Figura 9 – Branches do repositório no GitHub



10 CONSIDERAÇÕES FINAIS

O sistema desenvolvido atende aos requisitos propostos: realiza cadastro, login e logout com senha criptografada, controla o acesso por três perfis, armazena

usuários e sessões no MongoDB Atlas e mantém a interface separada da lógica de negócio. A organização em camadas e o uso de temas configuráveis permitem que o projeto seja reaproveitado como base de autenticação em outros sistemas, incluindo o PFC.

REFERÊNCIAS

MONGODB. MongoDB Atlas. Disponível em: <https://www.mongodb.com/products/platform>. Acesso em: 4 out. 2026.

SPRING. Spring Boot. Disponível em: <https://spring.io/projects/spring-boot>. Acesso em: 4 out. 2026.

SPRING. Spring Initializr. Disponível em: <https://start.spring.io>. Acesso em: 4 out. 2026.

SPRING. Spring Security. Disponível em: <https://spring.io/projects/spring-security>. Acesso em: 4 out. 2026.

THYMELEAF. Tutorial: Thymeleaf + Spring. Disponível em: <https://www.thymeleaf.org>. Acesso em: 4 out. 2026.